澳門理工大學
Universidade Politécnica de Macau
Macao Polytechnic University

Introduction to Data Manipulation

Jie Zhang
Faculty of Applied Sciences, Macao Polytechnic University
Email: jpeter.zhang@mpu.edu.mo
Personal Profile: easy-shu.github.io



Python Packages for Data Science

- Numpy and Scipy-fundamental scientific computing
- Pandas-data manipulation and analysis.
- Matplotlib, Seaborn, and Plotnine - plotting and visualization.
- Scikit-learn- machine learning and data mining
- StatsModels-statistical modeling, testing, and analysis



pandas

About us | Getting started | Documentation | Community | Contribute

pandas

pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language.

Install pandas now!

Getting started

- Install pandas
- Getting started

Documentation

- User guide
- API reference
- Contributing to pandas
- Release notes

Community

- About pandas
- Ask a question
- Ecosystem

With the support of:

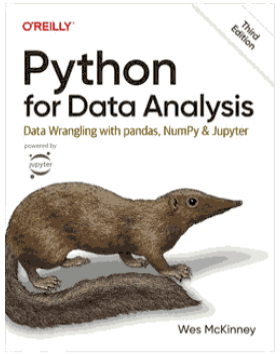
NUMFOCUS OPEN CODE = BETTER SCIENCE
TWO SIGMA
VOLTRON DATA
Coiled A Dask Company
Quansight Labs
NVIDIA
intel
TIDELIFT
Chan Zuckerberg Initiative
bodo.ai

Latest version: 2.2.3

- What's new in 2.2.3
- Release date: Sep 20, 2024
- Documentation (web)
- Download source code

Follow us

Recommended books





https://pandas.pydata.org/

Outline

1. DataFrame Creation
2. DataFrame Reading and Writing
3. DataFrame Indexing and Selection
4. Categorical Data in DataFrame
5. DataFrame Sorting
6. DataFrame Transformation
7. DataFrame Concatenation and Merging
8. DataFrame Aggregation and Grouping
9. Handling Duplicate and Missing Data
10. Vectorized String and Time Series Operations



Pandas

The Pandas library is built on NumPy and provides easy-to-use data structures and data analysis tools for the Python programming language.

pandas can be installed via pip from PyPI: pip install pandas

Use the following import convention: import pandas as pd

Once Pandas is installed, you can import it and check the version:

```
In[1]: import pandas
       pandas.__version__

Out[1]: '0.18.1'
```



DataFrame Creation

DataFrame (data frame) is similar to an Excel spreadsheet and also similar to the DataFrame data structure in the R language. There are many ways to create instances of a class similar to DataFrame, including the following methods:

(1) Creating a DataFrame using a list or ndarray object:

```
df=pd.DataFrame([['a', 1, 2], ['b', 2, 5], ['c', 3, 3]], columns=['x','y','z'])
```

```
df=pd.DataFrame(np.zeros((3,3)), columns=['x','y','z'])
```

Columns

Index	x	y	z
0	a	1	2
1	b	2	5
2	c	3	3

A two-dimensional labeled data structure with columns of potentially different types

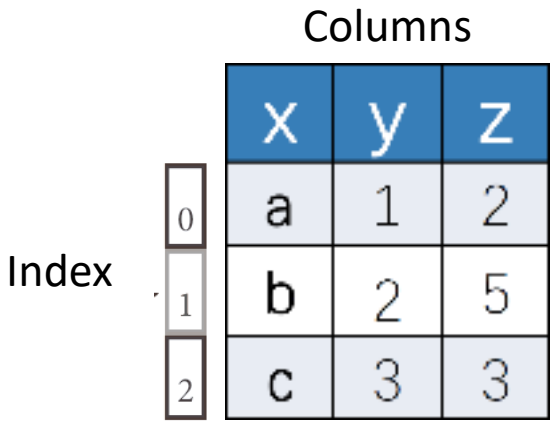
(2) Creating a DataFrame using a dictionary: When creating a DataFrame instance using a dictionary, you can directly set the keys of the dictionary as column indexes in the DataFrame. You can specify a list as the dictionary's values, and these values become all the elements under that column index.

```
df=pd.DataFrame({'x': ['a', 'b','c'],'y':range(1,4), 'z':[2,5,3]})
```

```
df=pd.DataFrame(dict(x=['a', 'b','c'],y=range(1,4), z=[2,5,3]))
```




DataFrame Creation



A two-dimensional labeled data structure with columns of potentially different types

- To get the number of rows, columns, and dimensions of a DataFrame: df.shape[0] or len(df), df.shape[1], df.shape.
- To get the column names or row names of a DataFrame: df.columns, df.index.
- To redefine column names: df.columns = ["X", "Y", "Z"].
- To rename a specific column: df.rename(columns={'x':'X'}, inplace=True). Note that without the inplace option, the changes will not be made; instead, a new column will be added.
- Observing the contents of a DataFrame:
  - df.info(): The info attribute prints the attribute information of the DataFrame.
  - df.head(): View the first five rows of data in the DataFrame.
  - df.tail(): View the last five rows of data in the DataFrame.

DataFrame Creation

Basic Information

`>>> df.shape`	(rows,columns)
`>>> df.index`	Describe index
`>>> df.columns`	Describe DataFrame columns
`>>> df.info()`	Info on DataFrame
`>>> df.count()`	Number of non-NA values

Summary

`>>> df.sum()`	Sum of values
`>>> df.cumsum()`	Cummulative sum of values
`>>> df.min()/df.max()`	Minimum/maximum values
`>>> df.idxmin()/df.idxmax()`	Minimum/Maximum index value
`>>> df.describe()`	Summary statistics
`>>> df.mean()`	Mean of values
`>>> df.median()`	Median of values

Columns

Index

	x	y	z
0	a	1	2
1	b	2	5
2	c	3	3

A two-dimensional labeled data structure with columns of potentially different types



DataFrame Creation: Empty DataFrame

Creating an empty DataFrame is particularly important when you need to construct data information for plotting your own charts.

Sometimes, when drawing complex data visualizations, you may need to perform operations like interpolation, fitting, etc., on existing data before storing the new data in an empty DataFrame and using that new DataFrame to plot the charts.

```
df_empty= pd.DataFrame(columns=['x','y','z'])
```

```
# Create an empty DataFrame
df = pd.DataFrame(columns=['Country', 'Population'])

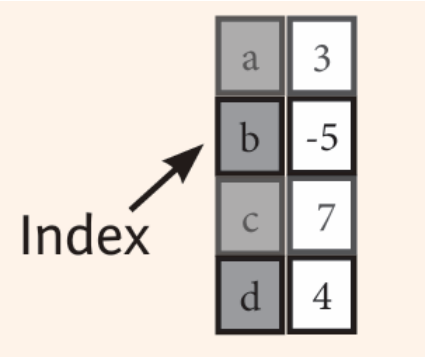
# Populate the DataFrame with data
data = pd.DataFrame( {'Country': ['USA', 'China', 'India', 'Brazil'], 'Population': [331, 1441, 1393, 212]} )

df = pd.concat([df, data], ignore_index=True)
```



DataFrame Creation: Series

Series is a one-dimensional labeled array capable of holding any data type.



Statement	Output	Statement	Output
s=pd.Series([1,3,2,4])		s=pd.Series([1,3,2,4], index=['a', 'b','c','d'])	
s.values	[1,3,2,4]	s.values	[1,3,2,4]
s.index	[0,1,2,3]	s.index	['a', 'b','c','d']

DataFrame Reading and Writing

```
# Sample DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie', 'Diana’],
        'Age': [25, 30, 28, 35],
        'Gender': ['Female', 'Male', 'Male', 'Female’]}
df = pd.DataFrame(data)
```

DataFrame Reading and Writing CSV file

```
# Save DataFrame to CSV and Excel files
df.to_csv('data_saved.csv', index=False)
# Read the saved CSV  files back into DataFrames
df_csv = pd.read_csv('data_saved.csv')
```

DataFrame Reading and Writing Excel file

```
# Save DataFrame to CSV and Excel files
df.to_excel('data_saved.xlsx', index=False)
# Read the saved Excel  files back into DataFrames
df_excel = pd.read_excel('data_saved.xlsx')
```




DataFrame Reading and Writing: Excel

pd.read_excel()

function:==========

- sheet_name specifies the sheet page, defaulted to 0; header specifies the row with column names, defaulted to 0, meaning it takes the first row as column names, with data below this row. If the data lacks column names, set header = None.

```
df = pd.read_excel("data.xlsx", sheet_name='sheet_name', header=0)
```

to_excel() function:==========

- excel_writer indicates the target path; index=False disregards the index column.

```
df.to_excel(excel_writer, sheet_name='sheet_name', index=False)
```




DataFrame Reading and Writing: CSV

pd.read_csv()

function:==========

- It is crucial to set the encoding parameter based on the encoding format of the data file, such as utf-8, ansi, and gbk, especially when the data contains Chinese characters.
- The delimiter parameter specifies the separator between columns in the data file. Delimiters longer than one character are treated as regular expressions, such as one or more spaces (\s+), tab characters (\t), and so on.

```
df = pd.read_csv("Data.csv", sep=",", header=0, index_col=None, encoding="utf-8")
```

to_csv() function:==========

- index=False indicates that index information should be ignored, while index=True indicates that the first column of the output file should retain index values.

```
df.to_csv("Data.csv", index=False, header=True)
```

CSV files are simple, convertible with Excel, have smaller sizes than Excel, store data efficiently, and offer basic security without explanatory data. CSV is recommended for data import/export.



DataFrame Reading and Writing: SQL Query

This code snippet demonstrates how to create an in-memory SQLite database, write a DataFrame to an SQLite table, and then read data from the SQLite table using pandas and SQLAlchemy in Python.

```
import pandas as pd
from sqlalchemy import create_engine
# Sample Data
data = { 'Name': ['Alice', 'Bob', 'Charlie', 'Diana'], 'Age': [25, 30, 28, 35], 'Gender': ['Female', 'Male', 'Male', 'Female'] }
df = pd.DataFrame(data)

# Create an in-memory SQLite engine
engine = create_engine('sqlite:///:memory:')
# Write the DataFrame to an SQL table
df.to_sql('my_table', con=engine, index=False)

# Read data from the SQL table using SQL query
query = "SELECT * FROM my_table WHERE Age > 28"
data_from_query = pd.read_sql_query(query, con=engine)
data_from_sql_table = pd.read_sql_table('my_table', engine)
```




DataFrame Indexing and Selection

Statement	Output
Create a Dataframe: df=pd.DataFrame({'x': ['a', 'b','c'], 'y':range(1,4), 'z':[2,5,3]})	x y z a 1 2 b 2 5 c 3 3
Select the 1rst and 2nd columns: df[['x','y']] df.loc[:,['x','y']] df.iloc[:,[0,1]]	x y z a 1 2 b 2 5 c 3 3
Select the 1rst and 2nd rows: df.loc[[0,1],:] df.iloc[[0,1],:]	x y z a 1 2 b 2 5 c 3 3
Filter array with a single condition: df[df.z>=3]	x y z a 1 2 b 2 5 c 3 3

Statement	Output
Select the 2nd column: df['y'] df.y df.loc[:,['y']] df.iloc[:,[1]]	x y z a 1 2 b 2 5 c 3 3
Select the 2nd row: df.loc[1,:] df.iloc[1,:]	x y z a 1 2 b 2 5 c 3 3
Select the orange number: df.loc[1,'y'] df.loc[[1],['y']] df.iloc[1,1]	x y z a 1 2 b 2 5 c 3 3
Filter array with multiple conditions: df[(df.z>=3) & (df.z<=4)] df.query('z>=3 & z<=4')	x y z a 1 2 b 2 5 c 3 3

Columns

Index

	x	y	z
0	a	1	2
1	b	2	5
2	c	3	3

A two-dimensional labeled data structure with columns of potentially different types



DataFrame Indexing and Selection: MultiIndexing

In a DataFrame, typically there is a single-level index, but there are cases where a multi-level index is required. When using df.set_index(['X', 'year']), which sets two levels of index (with "X" as the 0th level index and "year" as the 1st level index), you can utilize the loc method to select data based on these multi-level indices.

```
df=pd.DataFrame(dict( X=['A','B','C','A','B','C'],
                      year=[2010,2010,2010, 2011,2011,2011],
                     Value=[1,3,4,3,5,2]))
```

index	X	year	Value
0	A	2010	1
1	B	2010	3
2	C	2010	4
3	A	2011	3
4	B	2011	5
5	C	2011	2

Setting the index of the DataFrame df to ['X', 'year']:
df=df.set_index(['X','year'])

		Value
X	year	
A	2010	1
B	2010	3
C	2010	4
A	2011	3
B	2011	5
C	2011	2



DataFrame Indexing and Selection: MultiIndexing

In a DataFrame, typically there is a single-level index, but there are cases where a multi-level index is required. When using df.set_index(['X', 'year']), which sets two levels of index (with "X" as the 0th level index and "year" as the 1st level index), you can utilize the loc method to select data based on these multi-level indices.

```
df=pd.DataFrame(dict( X=['A','B','C','A','B','C'],
                      year=[2010,2010,2010, 2011,2011,2011],
                     Value=[1,3,4,3,5,2]))
```

index	X	year	Value
0	A	2010	1
1	B	2010	3
2	C	2010	4
3	A	2011	3
4	B	2011	5
5	C	2011	2

Setting the index of the DataFrame df to ['X', 'year']:

```
df=df.set_index(['X','year'])
```

		Value
X	year	
A	2010	1
B	2010	3
C	2010	4
A	2011	3
B	2011	5
C	2011	2

Selecting a specific element where 'A' corresponds to the level 0 index, 2010 corresponds to the level 1 index, and 'Value' is the specified column in the DataFrame df:

```
df.loc[('A', 2010), 'Value']
```

		Value
X	year	
A	2010	1
B	2010	3
C	2010	4
A	2011	3
B	2011	5
C	2011	2



DataFrame Indexing and Selection: MultiIndexing

In a DataFrame, typically there is a single-level index, but there are cases where a multi-level index is required. When using df.set_index(['X', 'year']), which sets two levels of index (with "X" as the 0th level index and "year" as the 1st level index), you can utilize the loc method to select data based on these multi-level indices.

```
df=pd.DataFrame(dict( X=['A','B','C','A','B','C'],
                      year=[2010,2010,2010, 2011,2011,2011],
                     Value=[1,3,4,3,5,2]))
```

index	X	year	Value
0	A	2010	1
1	B	2010	3
2	C	2010	4
3	A	2011	3
4	B	2011	5
5	C	2011	2

Setting the index of the DataFrame df to ['X', 'year']:

```
df=df.set_index(['X','year'])
```

		Value
X	year	
A	2010	1
B	2010	3
C	2010	4
A	2011	3
B	2011	5
C	2011	2

Selecting all elements of a particular category under the 0th level index, where slice(None) is used for slicing operations to select any ID. Please note that you cannot use a colon ':' to specify any index:

```
df.loc[('A',slice(None)),'Value']
```

		Value
X	year	
A	2010	1
B	2010	3
C	2010	4
A	2011	3
B	2011	5
C	2011	2



DataFrame Indexing and Selection: MultiIndexing

In a DataFrame, typically there is a single-level index, but there are cases where a multi-level index is required. When using df.set_index(['X', 'year']), which sets two levels of index (with "X" as the 0th level index and "year" as the 1st level index), you can utilize the loc method to select data based on these multi-level indices.

```
df=pd.DataFrame(dict( X=['A','B','C','A','B','C'],
                      year=[2010,2010,2010, 2011,2011,2011],
                     Value=[1,3,4,3,5,2]))
```

index	X	year	Value
0	A	2010	1
1	B	2010	3
2	C	2010	4
3	A	2011	3
4	B	2011	5
5	C	2011	2



DataFrame Indexing and Selection: Index Manipulation

- **Reset Index:** The reset_index() function is used to reset the index of a DataFrame. By default, it converts the current index into a column and creates a new integer index.

```
df.reset_index(drop=True, inplace=True)
```

- **Set Index**: The set_index() function is employed to set a specific column as the index of the DataFrame.

```
df.set_index('column_name', inplace=True)
```

```
# Sample DataFrame
df = pd.DataFrame({'A': [4,5,3,2], 'B': ['apple', 'banana', 'cherry', 'date']})

# Set Index
df.set_index('A', inplace=True)

# Reset Index
df_reset = df.reset_index(drop=True)
```

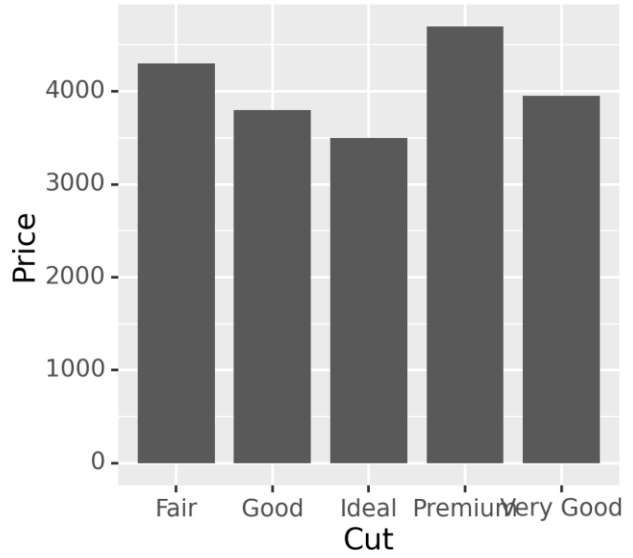



Categorical Data in DataFrame: Application Scenarios

```
import pandas as pd
from plotnine import *

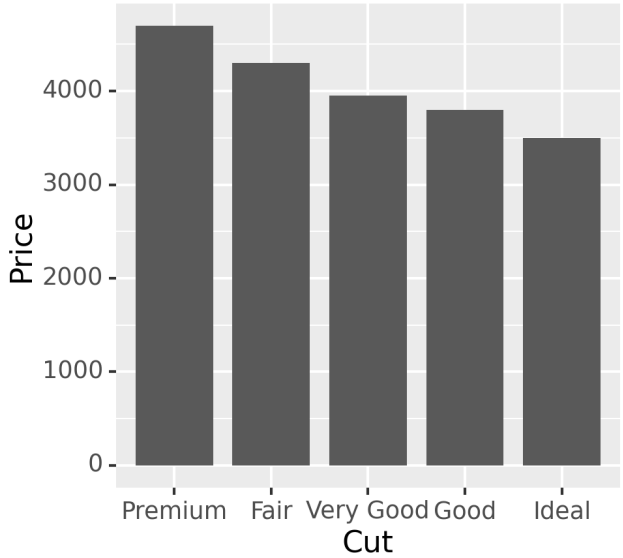
mydata = pd.DataFrame({'Cut': ["Fair", "Good", "Very Good", "Premium",
"Ideal"], 'Price': [4300, 3800, 3950, 4700, 3500]})

(ggplot(mydata , aes(x='Cut', y='Price'))+
        geom_bar(stat="identity", width=0.8)+
         theme(figure_size=(3.2,3),dpi=300))
```




```
Sort_data = mydata.sort_values(by='Price', ascending=False)

(ggplot(Sort_data , aes(x='Cut', y='Price'))+
        geom_bar(stat="identity", width=0.8)+
         theme(figure_size=(3.2,3),dpi=300))
```





Categorical Data in DataFrame: Application Scenarios

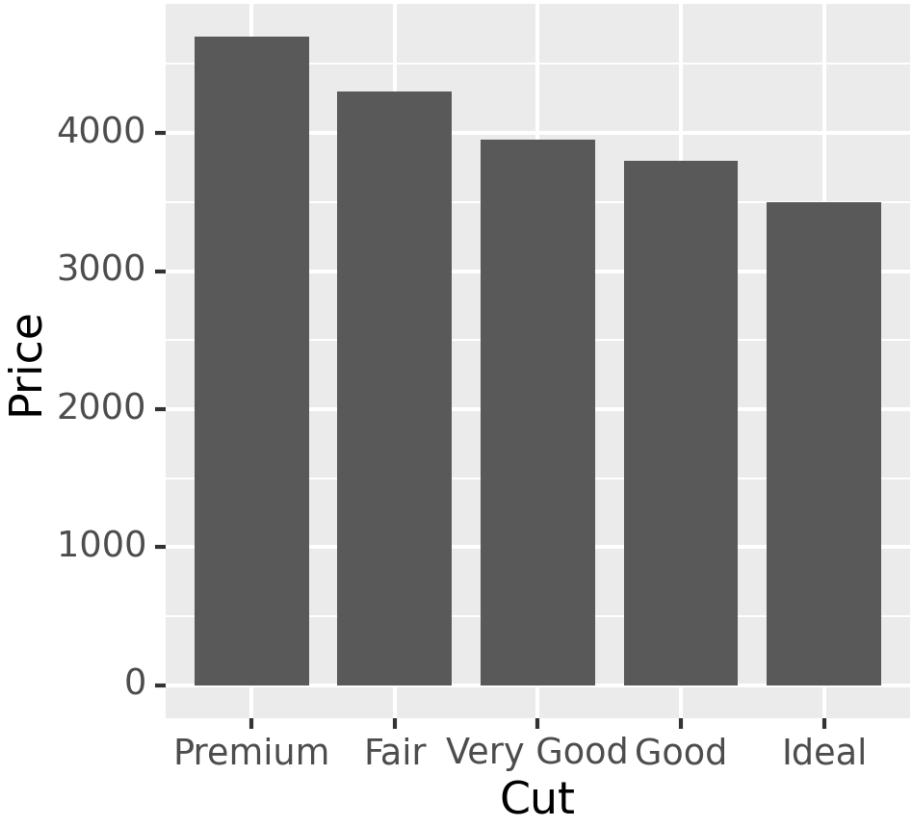
```
import pandas as pd
from plotnine import *
from pandas.api.types import CategoricalDtype

mydata = pd.DataFrame({'Cut': ["Fair", "Good", "Very Good", "Premium",
"Ideal"], 'Price': [4300, 3800, 3950, 4700, 3500]})

Sort_data = mydata.sort_values(by='Price', ascending=False)

Sort_data['Cut'] = Sort_data['Cut'].astype(CategoricalDtype(categories
=Sort_data['Cut'].values,ordered=True))

(ggplot(Sort_data, aes(x='Cut', y='Price'))+
        geom_bar(stat="identity", width=0.8)+
         theme(figure_size=(3.2,3),dpi=300))
```



Categorical Data in DataFrame: Categorical Data Creation

- Pandas has a special data structure type called Categorical, which can be used to hold data that represents categories based on integers. **Categorical data can be divided into categorical and ordinal types**, similar to factor vectors in the R language.
- Categorical data types can be seen as lists that contain additional information, which is the different categories, often referred to as "categories.
- In Python, the categorical data type is crucial in the plotnine package as it determines how data is analyzed and presented visually.

Categorical data, which includes the categorical variables and potential categories, can be managed using the pd.Categorical() function. **By default, when working with character types, categories are arranged alphabetically**: [Fair, Good, Ideal, Premium, Very Good].

```
Cut = ["Fair", "Good", "Very Good", "Premium", "Ideal"]
Cut_Factor1 = pd.Categorical(Cut)
```

If the default alphabetical order is unsatisfactory, **custom categories can be defined to replace the default sorting**. For instance, changing the categories to [Good, Fair, Very Good, Ideal, Premium] can be achieved directly within the pd.Categorical() function.

```
Cut_Factor2 = pd.Categorical(["Fair", "Good", "Very Good", "Premium", "Ideal"],
                             categories=["Good", "Fair", "Very Good", "Ideal", "Premium"], ordered=True)
```



Categorical Data in DataFrame: Categorical Data Modification

For existing categorical data or data frames, you can use the *.astype() function to specify category options and override the default sorting, thereby changing the categories of the categorical data to [Good, Fair, Very Good, Ideal, Premium].

With the update to Pandas version 1.0.3, the original method for changing categories requires using pandas.api.types.CategoricalDtype to define categories. When ordered=True, the categories are ordered as [Good < Fair < Very Good < Ideal < Premium].

```
from pandas.api.types import CategoricalDtype
Cut = pd.Series(["Fair", "Good", "Very Good", "Premium", "Ideal"])
Cut_Factor2 = Cut.astype(CategoricalDtype(categories=["Good", "Fair", "Very Good", "Ideal", "Premium"], ordered=True))
```




Categorical Data in DataFrame: Categorical Data Transformation

We need to obtain the categories and codes of categorical data. This is akin to converting categorical data into numerical data.

```
Cut = ["Fair", "Good", "Very Good", "Premium", "Ideal"]
Cut_Factor1 = pd.Categorical(Cut)
```

```
Cut_Factor2 = pd.Categorical(["Fair", "Good", "Very Good", "Premium", "Ideal"],
                             categories=["Good", "Fair", "Very Good", "Ideal", "Premium"], ordered=True)
```

Statement	Output
Cut_Facor1.codes	array([0, 1, 4, 3, 2], dtype=int8)
Cut_Facor1.categories	Index(['Fair', 'Good', 'Ideal', 'Premium', 'Very Good'], dtype='object')
Cut_Facor2.array.codes	array([1, 0, 2, 4, 3], dtype=int8)
Cut_Facor2.array.categories	Index(['Fair', 'Good', 'Ideal', 'Premium', 'Very Good'], dtype='object')



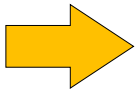
DataFrame Sorting

Use the sort_values() function to sort the entire table based on a column's values. Here, ascending=False indicates sorting the DataFrame df in descending order based on the 'value' colum.

```
data = pd.DataFrame ({ ‘X': ['A', 'B', 'C', 'A', 'B', 'C’],
          ‘year': ['2010', '2010', '2010', '2011', '2011', '2011’],
          'value': [1, 3, 4, 3, 5, 2] } )
dat_sort1= data.sort_values(by='value',ascending=True)
dat_sort2= data.sort_values(by='value',ascending=False)
dat_sort3= data.sort_values(by=['year','value'],ascending=True)
```

data

X	year	value
A	2010	1
B	2010	3
C	2010	4
A	2011	3
B	2011	5
C	2011	2



dat_sort1

X	year	value
A	2010	1
B	2011	2
C	2010	3
A	2011	3
B	2010	4
C	2011	5

dat_sort2

X	year	value
A	2011	5
B	2010	4
C	2011	3
A	2010	3
B	2011	2
C	2010	1

dat_sort3

X	year	value
A	2010	1
B	2010	3
C	2010	4
A	2011	2
B	2011	3
C	2011	5



https://pandas.pydata.org/docs/reference/api/pandas.melt.html

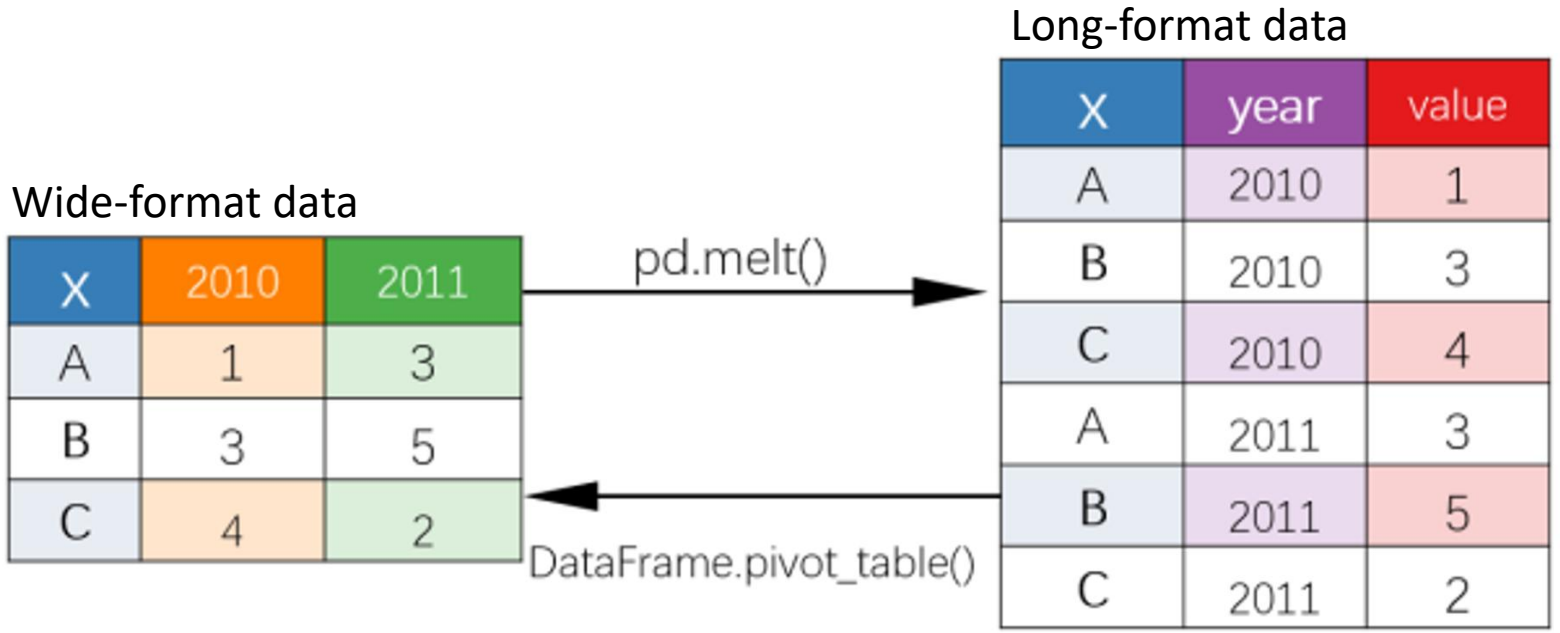
DataFrame Transformation

By converting wide data to long data, you can often make your data more accessible, easier to analyze, and better suited for various data processing tasks.

```
df = pd.DataFrame({'X': ['A', 'B', 'C'], '2010': [1, 3, 4], '2011': [3, 5, 2]})  #Wide-format data
```

To convert wide data to long data, aggregating multiple rows into columns and transforming a Wide-format table into a Long-format table

```
df_melt = pd.melt(df, id_vars='X', var_name='year', value_name='value')
```



https://pandas.pydata.org/docs/reference/api/pandas.pivot_table.html

DataFrame Transformation

To convert wide data to long data, aggregating multiple rows into columns and transforming a Wide-format table into a Long-format table

```
df_melt = pd.DataFrame({'X': ['A', 'B', 'C', 'A', 'B', 'C'], 'year': ['2010', '2010', '2010', '2011', '2011', '2011'], 'value': [1, 3, 4, 3, 5, 2]})
#df_melt = pd.melt(df, id_vars='X', var_name='year', value_name='value')
```

To convert long data back to wide data, expanding one column into multiple rows and transforming a one-dimensional table into a two-dimensional table:

```
df_pivot = pd.pivot_table(df_melt, index='X', columns='year', values='value')
df_pivot = df_pivot.reset_index()
```

Wide-format data

X	2010	2011
A	1	3
B	3	5
C	4	2

pd.melt()

DataFrame.pivot_table()

Long-format data

X	year	value
A	2010	1
B	2010	3
C	2010	4
A	2011	3
B	2011	5
C	2011	2



DataFrame Concatenation and Merging

```
df1=pd.DataFrame(dict(x= ["a","b","c"], y=range(1,4)))
df2=pd.DataFrame(dict(z= ["B","D","H"], g =[2,5,3]))
df3=pd.DataFrame(dict(x= ["g","d"], y =[2,5]))
```

```
dat_cbind=pd.concat([df1,df2],axis=1)

dat_rbind=pd.concat([df1,df3],axis=0)
```

df1

x	y
a	1
b	2
c	3

df2

z	g
B	2
D	5
H	3

df3

x	y
g	2
d	5

dat_cbind

x	y	z	g
a	1	B	2
b	2	D	5
c	3	H	3

dat_rbind

x	y
a	1
b	2
c	3
g	2
d	5



DataFrame Concatenation and Merging

Horizontal concatenation with keys:

```
# Sample data
s1= pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
s2 = pd.DataFrame({'C': [5, 6], 'D': [7, 8]})
# Horizontal concatenation with keys
result_horizontal = pd.concat([s1, s2], axis=1, keys=['One', 'Two'])
print(result_horizontal)
```

Vertical concatenation with inner join:

```
# Sample data
data1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
data2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
# Vertical concatenation with inner join
result_vertical = pd.concat([data1, data2], axis=1, join='inner')
print(result_vertical)
```




DataFrame Concatenation and Merging

To delete a column, such as the "y" column in df1, you can use the drop() function as follows:

```
df1=pd.DataFrame(dict(x= ["a","b","c"], y=range(1,4)))
df1.drop(labels="y",axis=1, inplace=True)
```

In this code snippet, columns specifies the column to be deleted, and inplace=True ensures that the operation is performed directly on the original DataFrame, modifying it permanently.

df1

x	y
a	1
b	2
c	3



DataFrame Concatenation and Merging

Two DataFrames may not align perfectly for direct concatenation. In such cases, they need a common key column as the basis for merging. The most commonly used function for merging tables is pd.merge() in Python.

```
df1=pd.DataFrame(dict(x= ["a","b","c"], y=range(1,4)))
df2=pd.DataFrame(dict(x= ["a","b","d"], z =[2,5,3]))
df3=pd.DataFrame(dict(g= ["a","b","d"], z =[2,5,3]))
df4=pd.DataFrame(dict(x= ["a","b","d"], y=[1,4,2],z =[2,5,3]))
```

df1

x	y
a	1
b	2
c	3

df2

x	z
a	2
b	5
d	3

df3

g	z
a	2
b	5
d	3

df4

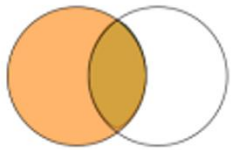
x	y	z
a	1	2
b	4	5
d	2	3

- To keep all data from the left table:

```
dat_merge1 = pd.merge(left=df1, right=df2, how="left", on="x")
```

dat_merge1

x	y	z
a	1	2
b	2	5
c	3	NA

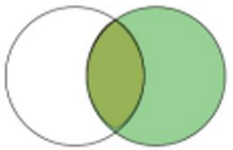


- To keep all data from the right table:

```
dat_merge2 = pd.merge(left=df1, right=df2, how="right", on="x")
```

dat_merge2

x	y	z
a	1	2
b	2	5
d	NA	3

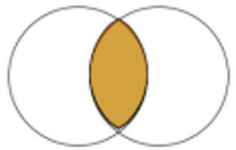


- To keep only the common information between the two tables:

```
dat_merge3 = pd.merge(left=df1, right=df2, how="inner", on="x")
```

dat_merge3

x	y	z
a	1	2
b	2	5

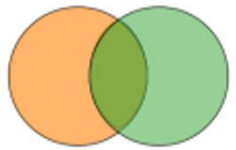


- To keep all information from both tables:

```
dat_merge4 = pd.merge(left=df1, right=df2, how="outer", on="x")
```

dat_merge4

x	y	z
a	1	2
b	2	5
c	3	NA
d	NA	3





DataFrame Concatenation and Merging

Two DataFrames may not align perfectly for direct concatenation. In such cases, they need a common key column as the basis for merging. The most commonly used function for merging tables is pd.merge() in Python.

```
df1=pd.DataFrame(dict(x= ["a","b","c"], y=range(1,4)))
df2=pd.DataFrame(dict(x= ["a","b","d"], z =[2,5,3]))
df3=pd.DataFrame(dict(g= ["a","b","d"], z =[2,5,3]))
df4=pd.DataFrame(dict(x= ["a","b","d"], y=[1,4,2],z =[2,5,3]))
```

df1

x	y
a	1
b	2
c	3

df2

x	z
a	2
b	5
d	3

df3

g	z
a	2
b	5
d	3

df4

x	y	z
a	1	2
b	4	5
d	2	3

- Using on=["x","y"] for matching on multiple columns:

```
dat_merge5 = pd.merge(left=df1, right=df4, how="left", on=["x","y"])
```

dat_merge5

x	y	z
a	1	2
b	2	NA
c	3	NA

- Merging based on different column names from the left and right tables:

```
dat_merge6 = pd.merge(left=df1, right=df3, how="left", left_on="x", right_on="g")
```

dat_merge6

x	y	z
a	1	2
b	2	5
c	3	NA

- When merging two tables with a column having the same name but different values, and you want to retain both values, you can use suffixes to add suffixes to the duplicate column names in each table:

```
dat_merge7 = pd.merge(left=df1, right=df4, how="left", on="x", suffixes=(".1", ".2"))
```

dat_merge7

x	y.1	y.2	z
a	1	1	2
b	2	4	5
c	3	NA	NA



DataFrame Aggregation and Grouping

Wide-format table: Row-wise and column-wise operations

df = pd.DataFrame({'x': ['A','B','C', 'A', 'C'],'2010':[1,3,4,4,3], '2011':[3,5,2,8,9]})

df **Wide-format table**

X	2010	2011
A	1	3
B	3	5
C	4	2
A	4	8
C	3	9

df_rowsum

Sum
4
8
6
12
12

df_colsum

2010	2011
15	27

1. **Row-wise Summation**:
df_rowsum = df[['2010', '2011']].sum(axis=1)
df_rowsum = df[['2010', '2011']].apply(lambda x: x.sum(), axis=1)

2.Column-wise Summation:
df_colsum = df[['2010', '2011']].sum(axis=0)
df_colsum = df[['2010', '2011']].apply(lambda x: x.sum(), axis=0)

3.Single Column Operation:
df['2010_2'] = df['2010']+2
df['2010_2'] = df['2010'].apply(lambda x: x + 2)
Here, a new column '2010_2' is created by adding 2 to each value in the '2010' column.

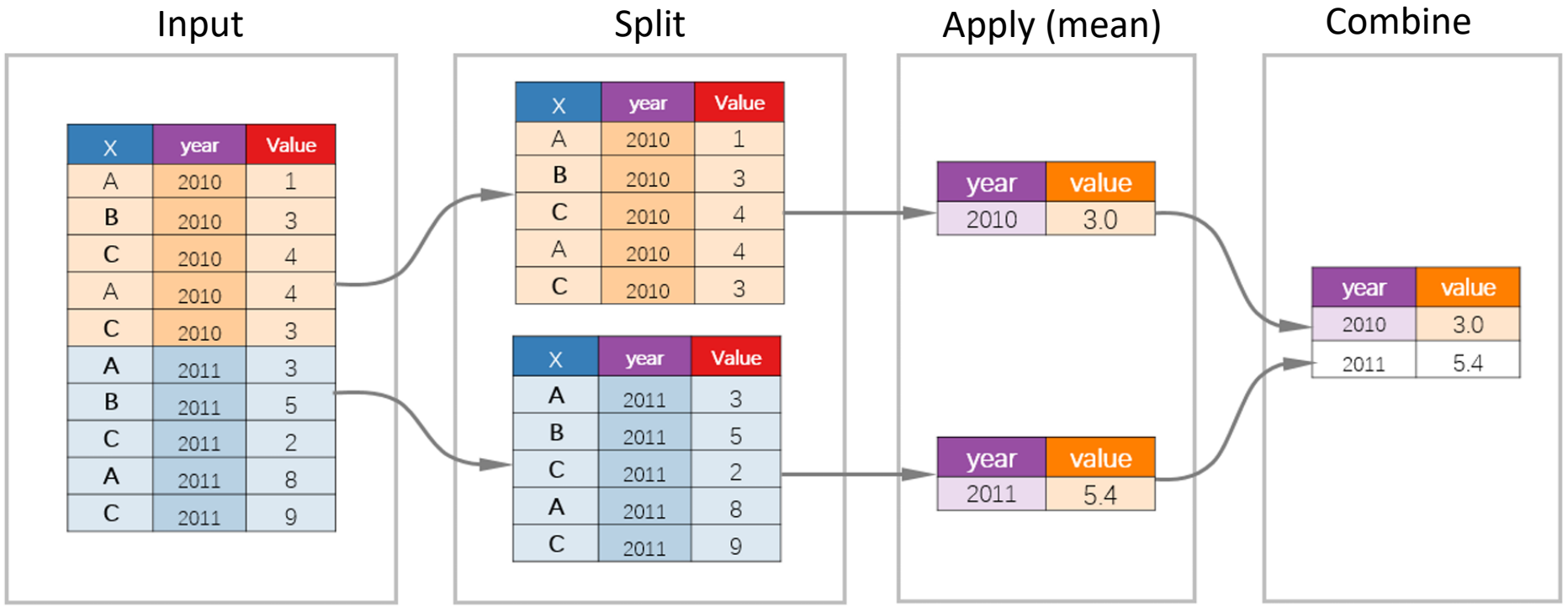
4.Multiple Column Operation:
df['2010_2011'] = df['2010'] + 2 * df['2011']
df['2010_2011'] = df.apply(lambda x: x['2010'] + 2 * x['2011'], axis=1)
This code snippet creates a new column '2010_2011' by performing an operation involving both the '2010' and '2011' columns for each row.

DataFrame Aggregation and Grouping

This process is commonly referred to as the "**split-apply-combine**" process:

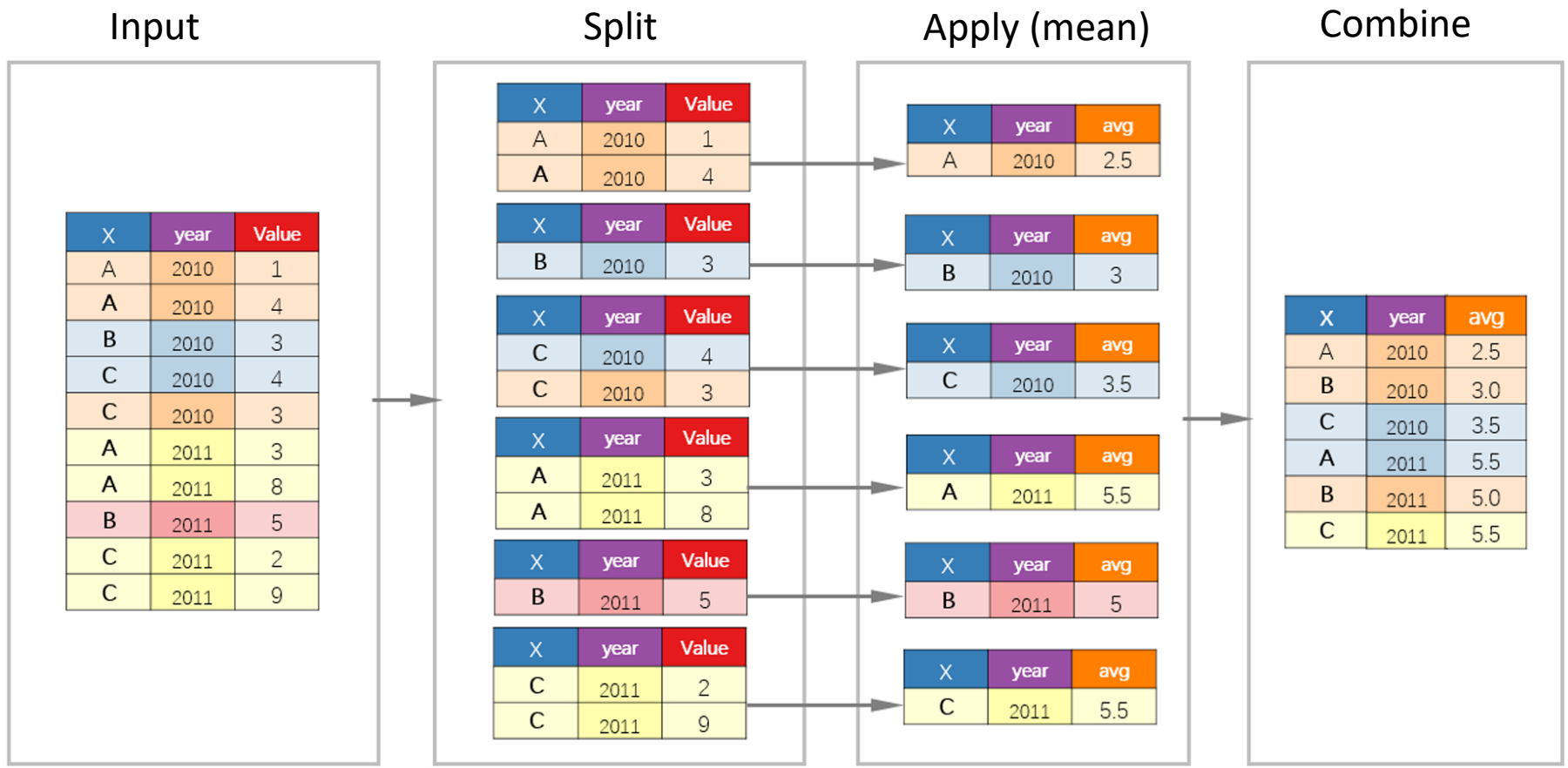
1. The split step involves breaking up and grouping a DataFrame depending on the value of the specified key.
2. The apply step involves computing some function, usually an aggregate, transformation, or filtering, within the individual groups.
3. The combine step merges the results of these operations into an output array.

```
df_melt = pd.DataFrame({'X': ['A', 'B', 'C', 'A', 'B', 'C'], 'year': ['2010', '2010', '2010', '2011', '2011', '2011'], 'value': [1, 3, 4, 3, 5, 2]})
# Grouping by 'year' and calculating the mean of 'value'
df_group_mean1=df_melt[["year","value"]].groupby('year').mean()
```



DataFrame Aggregation and Grouping

```
df_melt = pd.DataFrame({'X': ['A', 'B', 'C', 'A', 'B', 'C'], 'year': ['2010', '2010', '2010', '2011', '2011', '2011'], 'value': [1, 3, 4, 3, 5, 2]})
# Grouping by 'year' and 'X', and calculating the mean of 'value'
df_group_mean2=df_melt.groupby(['year','X'],as_index=False).mean()
```



DataFrame Aggregation and Grouping

```
df_melt = pd.DataFrame({'X': ['A', 'B', 'C', 'A', 'B', 'C'], 'year': ['2010', '2010', '2010', '2011', '2011', '2011'], 'value': [1, 3, 4, 3, 5, 2]})

#1。 Grouping by 'year' and calculating the sum:
df_group_sum = df_melt.groupby('year').sum()

#2. Grouping by 'year' and calculating the standard deviation:
df_group_std = df_melt.groupby('year').std()
```

Table 3-3. Listing of Pandas aggregation methods

Aggregation	Description
`count()`	Total number of items
`first(), last()`	First and last item
`mean(), median()`	Mean and median
`min(), max()`	Minimum and maximum
`std(), var()`	Standard deviation and variance
`mad()`	Mean absolute deviation
`prod()`	Product of all items
`sum()`	Sum of all items



DataFrame Aggregation and Grouping

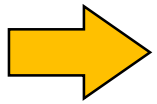
Dispatch methods.

Through some Python class magic, any method not explicitly implemented by the GroupBy object will be passed through and called on the groups, whether they are DataFrame or Series objects. For example, you can use the describe() method of DataFrames to perform a set of aggregations that describe each group in the data:

```
df_melt.groupby('X').describe().unstack()

df_melt.groupby('X')['year'].describe().unstack()
```

X	year	Value
A	2010	1
B	2010	3
C	2010	4
A	2010	4
C	2010	3
A	2011	3
B	2011	5
C	2011	2
A	2011	8
C	2011	9



```
                X
value  count    A    2.000000
                B    2.000000
                C    2.000000
       mean     A    2.000000
                B    4.000000
                C    3.000000
       std      A    1.414214
                B    1.414214
                C    1.414214
       min      A    1.000000
                B    3.000000
                C    2.000000
       25%      A    1.500000
                B    3.500000
                C    2.500000
       50%      A    2.000000
                B    4.000000
                C    3.000000
       75%      A    2.500000
                B    4.500000
                C    3.500000
       max      A    3.000000
                B    5.000000
                C    4.000000
dtype: float64
```

```
              X
count         A       2
              B       2
              C       2
unique        A       2
              B       2
              C       2
top           A    2010
              B    2010
              C    2010
freq          A       1
              B       1
              C       1
dtype: object
```



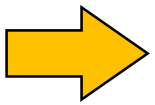
DataFrame Aggregation and Grouping

Aggregate

We're now familiar with GroupBy aggregations with sum(), median(), and the like, but the aggregate() method allows for even more flexibility. It can take a string, a function, or a list thereof, and compute all the aggregates at once.

```
df_group1=df_melt.groupby(['x','year']).aggregate({ np.mean, np.median})
df_group2=df_melt.groupby(['x','year']).agg(mean=('value','mean'), median=('value', 'median'))
df_group3=df_melt.groupby(['x','year'], as_index=False).agg(mean=('value','mean'), median=('value', 'median'))
```

X	year	Value
A	2010	1
B	2010	3
C	2010	4
A	2010	4
C	2010	3
A	2011	3
B	2011	5
C	2011	2
A	2011	8
C	2011	9



df_group1/df_group2

Index		value	value
x	year	mean	median
A	2010	2.5	2.5
A	2011	5.5	5.5
B	2010	3	3
B	2011	5	5
C	2010	3.5	3.5
C	2011	5.5	5.5

df_group3

Index	x	year	value	value
			mean	median
0	A	2010	2.5	2.5
1	A	2011	5.5	5.5
2	B	2010	3	3
3	B	2011	5	5
4	C	2010	3.5	3.5
5	C	2011	5.5	5.5



DataFrame Aggregation and Grouping

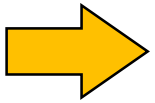
Transformation.

While aggregation must return a reduced version of the data, transformation can return some transformed version of the full data to recombine. For such a transformation, the output is the same shape as the input. A common example is to center the data by subtracting the group-wise mean:

```
(a) df_melt['percentage'] = df_melt.groupby('X')['value'].transform(lambda x: x / x.sum())

(b) df_melt['percentage'] = df_melt[["X","value"]].groupby('X').transform(lambda x: x / x.sum())
```

X	year	Value
A	2010	1
B	2010	3
C	2010	4
A	2010	4
C	2010	3
A	2011	3
B	2011	5
C	2011	2
A	2011	8
C	2011	9



```
   X  year  value  percentage
0  A  2010      1    0.250000
1  B  2010      3    0.375000
2  C  2010      4    0.666667
3  A  2011      3    0.750000
4  B  2011      5    0.625000
5  C  2011      2    0.333333
```



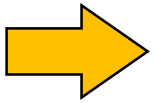

DataFrame Aggregation and Grouping

Filtering.

A filtering operation allows you to drop data based on the group properties. For example, we might want to keep all groups in which the standard deviation is larger than some critical value:

```
df_filter=df_melt.groupby('X').filter(lambda x: x['value'].mean()>2)
```

X	year	Value
A	2010	1
B	2010	3
C	2010	4
A	2010	4
C	2010	3
A	2011	3
B	2011	5
C	2011	2
A	2011	8
C	2011	9



```
   X  year  value
1  B  2010      3
2  C  2010      4
4  B  2011      5
5  C  2011      2
```



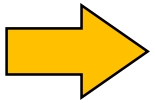

DataFrame Aggregation and Grouping

The apply() method.

The apply() method lets you apply an arbitrary function to the group results. The function should take a DataFrame, and return either a Pandas object (e.g., DataFrame, Series) or a scalar; the combine operation will be tailored to the type of output returned.

```
def norm_by_X(x):
    x['value'] /= x['value'].sum()
    return x
df_new=df_melt.groupby('X').apply(norm_by_X)
```

X	year	Value
A	2010	1
B	2010	3
C	2010	4
A	2010	4
C	2010	3
A	2011	3
B	2011	5
C	2011	2
A	2011	8
C	2011	9



		X	year	value
X				
A	0	A	2010	0.250000
	3	A	2011	0.750000
B	1	B	2010	0.375000
	4	B	2011	0.625000
C	2	C	2010	0.666667
	5	C	2011	0.333333



Handling Duplicate and Missing Data

Handling duplicate data is an important aspect of data preprocessing. In Python, particularly when working with pandas DataFrames, you can identify and handle duplicate data using various methods.

Handling Duplicate Data Example:

```
import numpy as np
df = pd.DataFrame({'x': ['a','b', 'a', 'd'],
                   'y':[ 1,   2,  1,   3]})

# Check for duplicate rows
duplicate_rows = df.duplicated()
# Drop duplicate rows
df_no_duplicates = df.drop_duplicates()
# Count the number of duplicate rows
num_duplicates = df.duplicated().sum()
```

1. Identifying Duplicate Rows:

- duplicated(): Returns a boolean Series indicating duplicate rows.
- drop_duplicates(): Removes duplicate rows.

2. Handling Duplicate Data:

(1) Identify Duplicate Rows:

```
# Check for duplicate rows based on all columns
duplicate_rows = df.duplicated()
# Check for duplicate rows based on specific columns
duplicate_rows_specific = df.duplicated(subset=['column1', 'column2'])
```

(2) Drop Duplicate Rows:

```
# Drop duplicate rows
df_no_duplicates = df.drop_duplicates()
```

(3) Count Duplicate Rows:

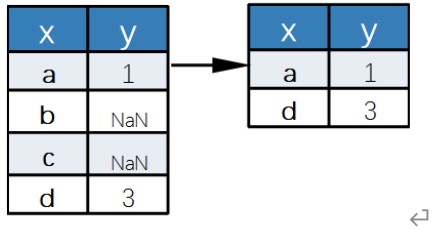
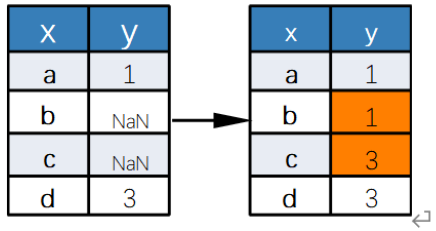
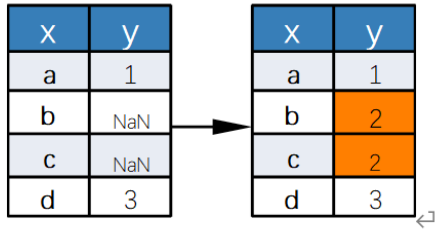
```
# Count the number of duplicate rows
num_duplicates = df.duplicated().sum()
```



Handling Duplicate and Missing Data

Handling missing data is a crucial aspect of data analysis and preprocessing in Python. The pandas library provides several methods to handle missing data in DataFrames.

```
import numpy as np
df = pd.DataFrame({'x': ['a','b','c', 'd'],
                   'y':[ 1, np.nan, np.nan,3]})
```

ID	Codes	Illustration
1	Delete rows with NaN directly: df_NA1=df.dropna(axis=0)	
2	Fill NaN with the nearest element: df_NA2=df.fillna(method="ffill")	
3	Replace NaN with the specified value: df_NA3=df.fillna(2)	

1. Detecting Missing Data:

- isnull(): Detect missing values in a DataFrame: df.isnull()
- notnull(): Inverse of isnull().

2. **Dealing with Missing Data**

(1) Dropping missing values:

- dropna(): Drop rows or columns with missing values.

(2) Filling missing values:

- fillna(): Fill missing values with a specified value or method.

(3) Interpolation:

- interpolate(): Interpolate missing values based on different methods like linear, quadratic, or cubic interpolation:
  df. interpolate(.

Vectorized String Operations

Nearly all Python's built-in string methods are mirrored by a Pandas vectorized string method. Here is a list of Pandas str methods that mirror Python string methods:

```
monte = pd.Series(['Graham Chapman', 'John Cleese', 'Terry Gilliam',
'Eric Idle', 'Terry Jones', 'Michael Palin'])
```

len()	lower()	translate()	islower()
ljust()	upper()	startswith()	isupper()
rjust()	find()	endswith()	isnumeric()
center()	rfind()	isalnum()	isdecimal()
zfill()	index()	isalpha()	split()
strip()	rindex()	isdigit()	rsplit()
rstrip()	capitalize()	isspace()	partition()
lstrip()	swapcase()	istitle()	rpartition()

```
In[7]: monte.str.lower()
Out[7]: 0    graham chapman
        1       john cleese
        2    terry gilliam
        3        eric idle
        4      terry jones
        5    michael palin
        dtype: object
```

```
In[8]: monte.str.len()
Out[8]: 0    14
        1    11
        2    13
        3     9
        4    11
        5    13
        dtype: int64
```

```
In[9]: monte.str.startswith('T')
Out[9]: 0    False
        1    False
        2     True
        3    False
        4     True
        5    False
        dtype: bool
```

```
In[10]: monte.str.split()
Out[10]: 0    [Graham, Chapman]
         1       [John, Cleese]
         2    [Terry, Gilliam]
         3        [Eric, Idle]
         4      [Terry, Jones]
         5    [Michael, Palin]
         dtype: object
```



Vectorized String Operations

Table 3-4. Mapping between Pandas methods and functions in Python

Method	Description
`match()`	Call `re.match()` on each element, returning a Boolean.
`extract()`	Call `re.match()` on each element, returning matched groups as strings.
`findall()`	Call `re.findall()` on each element.
`replace()`	Replace occurrences of pattern with some other string.
`contains()`	Call `re.search()` on each element, returning a Boolean.
`count()`	Count occurrences of pattern.
`split()`	Equivalent to `str.split()`, but accepts regexps.
`rsplit()`	Equivalent to `str.rsplit()`, but accepts regexps.

Table 3-5. Other Pandas string methods

Method	Description
`get()`	Index each element
`slice()`	Slice each element
`slice_replace()`	Replace slice in each element with passed value
`cat()`	Concatenate strings
`repeat()`	Repeat values
`normalize()`	Return Unicode form of string
`pad()`	Add whitespace to left, right, or both sides of strings
`wrap()`	Split long strings into lines with length less than a given width
`join()`	Join strings in each element of the `Series` with passed separator
`get_dummies()`	Extract dummy variables as a `DataFrame`

```
In[15]:
full_monte = pd.DataFrame({'name': monte,
                           'info': ['B|C|D', 'B|D', 'A|C', 'B|D', 'B|C',
                           'B|C|D']})
full_monte
```

```
Out[15]:     info            name
         0  B|C|D  Graham Chapman
         1    B|D     John Cleese
         2    A|C   Terry Gilliam
         3    B|D       Eric Idle
         4    B|C     Terry Jones
         5  B|C|D   Michael Palin
```

```
In[16]: full_monte['info'].str.get_dummies('|')
```

```
Out[16]:    A  B  C  D
         0  0  1  1  1
         1  0  1  0  1
         2  1  0  1  0
         3  0  1  0  1
         4  0  1  1  0
         5  0  1  1  1
```



Time Series Operations

Fundamental Pandas data structures for working with time series data:

- **Time stamps:** Pandas provides the Timestamp type. As mentioned before, it is essentially a replacement for Python's native datetime, but is based on the more efficient numpy.datetime64 data type. The associated index structure is **DatetimeIndex**.
- **Time period:** Pandas provides the Period type. This encodes a fixe frequency interval based on numpy.datetime64. The associated index structure I PeriodIndex.
- **Time deltas or duration:** Pandas provides the Timedelta type. Timedelta is a more efficient replacement for Python's native datetime.timedelta type, and is based on numpy.timedelta64. The associated index structure is TimedeltaIndex.

The most fundamental of these date/time objects are the Timestamp and DatetimeIn dex objects. While these class objects can be invoked directly, it is more common to use the pd.to_datetime() function, which can parse a wide variety of formats. Pass ing a single date to pd.to_datetime() yields a Timestamp; passing a series of dates by default yields a DatetimeIndex:

```
dates = pd.to_datetime([datetime(2015, 7, 3), '4th of July, 2015', '2015-Jul-6', '07-07-2015', '20150708'])
print(dates)
# DatetimeIndex(['2015-07-03', '2015-07-04', '2015-07-06', '2015-07-07', '2015-07-08'], dtype='datetime64[ns]', freq=None)
```



Time Series Operations

```
dates = pd.to_datetime([datetime(2015, 7, 3), '4th of July, 2015', '2015-Jul-6', '07-07-2015', '20150708’])
print(dates)
# DatetimeIndex(['2015-07-03', '2015-07-04', '2015-07-06', '2015-07-07', '2015-07-08'], dtype='datetime64[ns]', freq=None)
```

Any DatetimeIndex can be converted to a PeriodIndex with the to_period() func tion with the addition of a frequency code; here we’ll use 'D' to indicate daily frequency:

```
dates_D=dates.to_period(‘D‘)
#PeriodIndex(['2015-07-03', '2015-07-04', '2015-07-06', '2015-07-07', '2015-07-08'], dtype='period[D]')
```

A TimedeltaIndex is created, for example, when one date is subtracted from another:

```
dates_s=dates - dates[0]
# TimedeltaIndex(['0 days', '1 days', '3 days', '4 days', '5 days'], dtype='timedelta64[ns]', freq=None)
```




Time Series Operations

To make the creation of regular date sequences more convenient, Pandas offers a few functions for this purpose: pd.date_range() for timestamps, pd.period_range() for periods, and pd.timedelta_range() for time deltas.

```
In[18]: pd.date_range('2015-07-03', '2015-07-10')

Out[18]: DatetimeIndex(['2015-07-03', '2015-07-04', '2015-07-05', '2015-07-06',
                        '2015-07-07', '2015-07-08', '2015-07-09', '2015-07-10'],
                       dtype='datetime64[ns]', freq='D')

In[19]: pd.date_range('2015-07-03', periods=8)

Out[19]: DatetimeIndex(['2015-07-03', '2015-07-04', '2015-07-05', '2015-07-06',
                        '2015-07-07', '2015-07-08', '2015-07-09', '2015-07-10'],
                       dtype='datetime64[ns]', freq='D')

In[20]: pd.date_range('2015-07-03', periods=8, freq='H')

Out[20]: DatetimeIndex(['2015-07-03 00:00:00', '2015-07-03 01:00:00',
                        '2015-07-03 02:00:00', '2015-07-03 03:00:00',
                        '2015-07-03 04:00:00', '2015-07-03 05:00:00',
                        '2015-07-03 06:00:00', '2015-07-03 07:00:00'],
                       dtype='datetime64[ns]', freq='H')
```




Time Series Operations

To create regular sequences of period or time delta values, the very similar pd.period_range() and pd.timedelta_range() functions are useful.

```
In[21]: pd.period_range('2015-07', periods=8, freq='M')

Out[21]:
PeriodIndex(['2015-07', '2015-08', '2015-09', '2015-10', '2015-11', '2015-12',
             '2016-01', '2016-02'],
            dtype='int64', freq='M')
```

```
In[22]: pd.timedelta_range(0, periods=10, freq='H')

Out[22]:
TimedeltaIndex(['00:00:00', '01:00:00', '02:00:00', '03:00:00', '04:00:00',
                '05:00:00', '06:00:00', '07:00:00', '08:00:00', '09:00:00'],
               dtype='timedelta64[ns]', freq='H')
```

Table 3-7. Listing of Pandas frequency codes

Code	Description	Code	Description
D	Calendar day	B	Business day
W	Weekly		
M	Month end	BM	Business month end
Q	Quarter end	BQ	Business quarter end
A	Year end	BA	Business year end
H	Hours	BH	Business hours
T	Minutes		
S	Seconds		
L	Milliseonds		
U	Microseconds		
N	Nanoseconds		

Listing of start-indexed frequency codes

Code	Description
MS	Month start
BMS	Business month start
QS	Quarter start
BQS	Business quarter start
AS	Year start
BAS	Business year start



Time Series Operations

```
# Create a DataFrame from the provided data
data = { 'Date': ['2016-03-01', '2016-03-02', '2016-03-01', '2016-03-03', '2016-03-02', '2016-03-03'],
         'Type': ['a', 'b', 'c', 'a', 'a', 'c'],
         'Value': [11.432, 13.031, 20.784, 99.906, 1.303, 20.78] }
df2 = pd.DataFrame(data)

# Convert 'Date' column to datetime format
df2['Date'] = pd.to_datetime(df2['Date'])

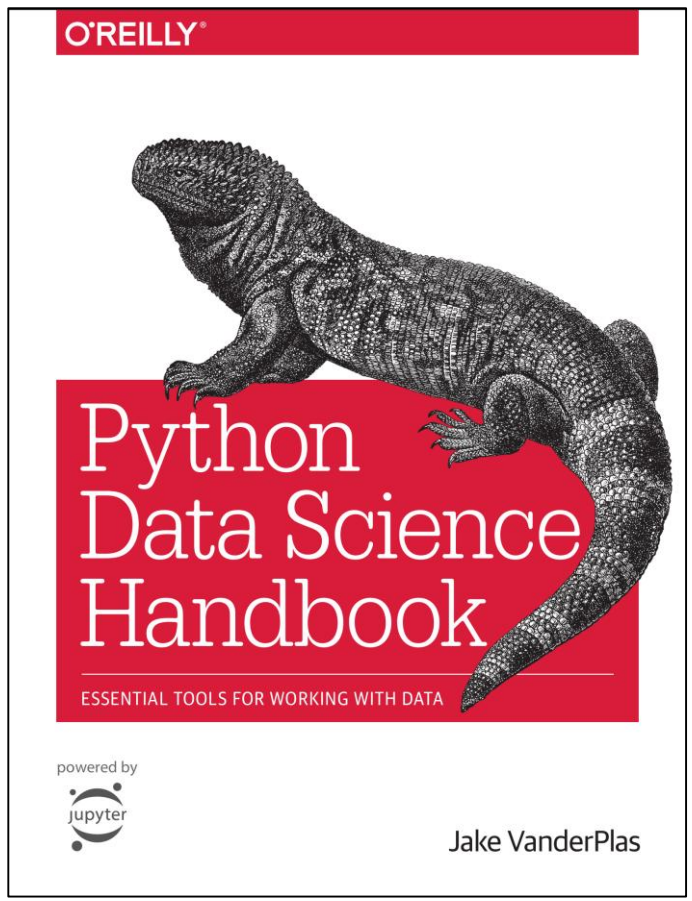
# Extract day, month, and year into separate columns
df2['Day'] = df2['Date'].dt.day
df2['Month'] = df2['Date'].dt.month
df2['Year'] = df2['Date'].dt.year

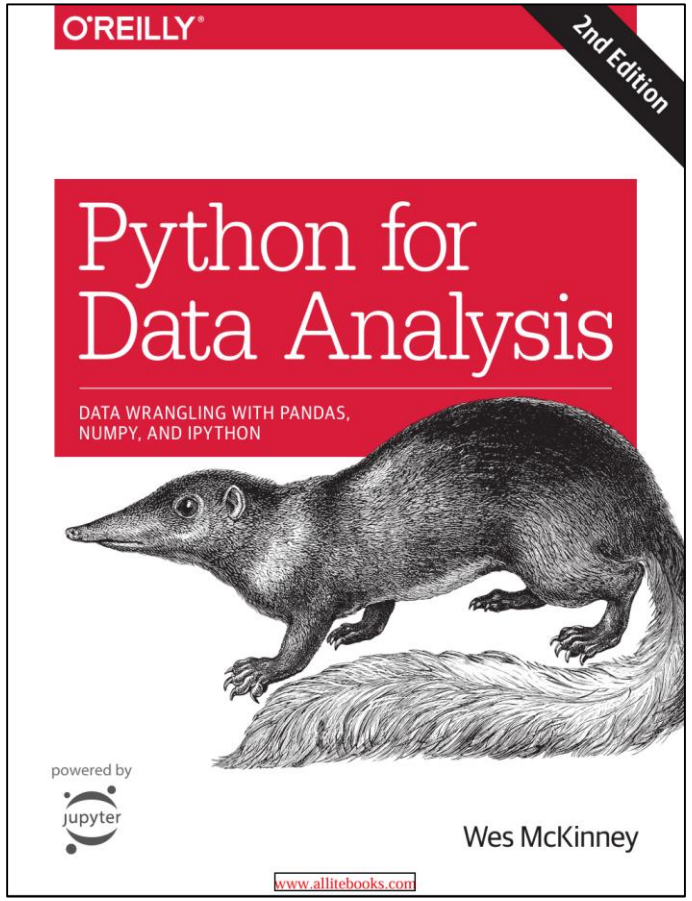
# Display the DataFrame with day, month, and year columns
print(df2)
```

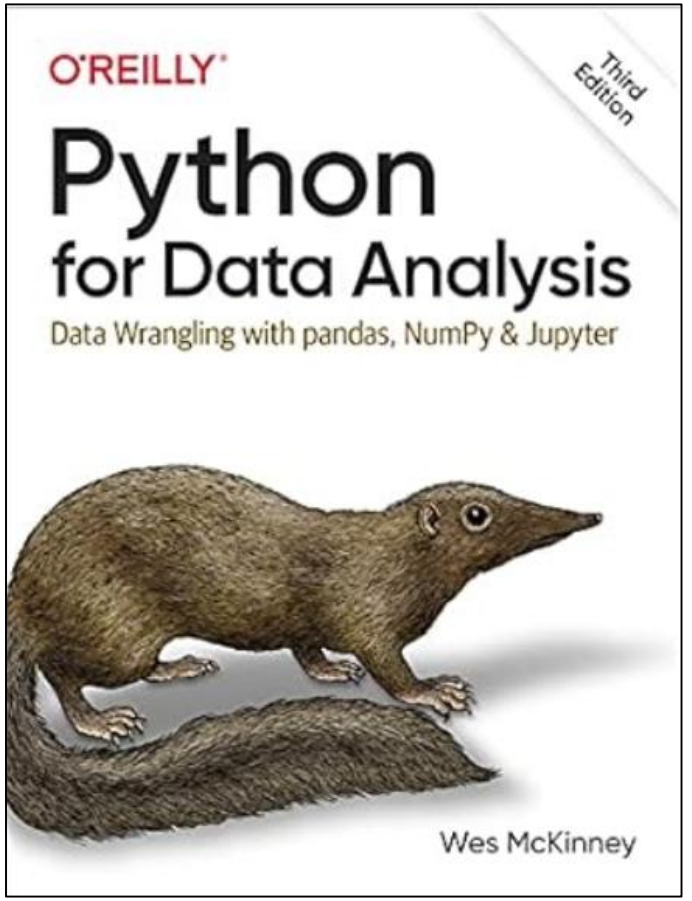
```
        Date Type   Value  Day  Month  Year
0 2016-03-01    a  11.432    1      3  2016
1 2016-03-02    b  13.031    2      3  2016
2 2016-03-01    c  20.784    1      3  2016
3 2016-03-03    a  99.906    3      3  2016
4 2016-03-02    a   1.303    2      3  2016
5 2016-03-03    c  20.780    3      3  2016
```




Extended Reading and Learning







Thanks

